D Algorithm Tracing on 4-bit Dadda Multiplier

Team Members:
Sangeerthana (2023BEC0044)
Amruth (2023BEC0050)

Faculty Mentor: Dr. Lakshmi NS

February 26, 2026

Contents

1 Overview of Dadda Multiplier

The Dadda multiplier is a parallel multiplier that seeks to minimize the number of full adders and half adders used in the partial product reduction stages.

The process involves three main steps:

1. Partial Product Generation: Create a matrix of ANDed bits.
2. Partial Product Reduction: Using a "Dadda Tree" of Full Adders and Half Adders to reduce the height of the matrix to exactly two rows.
3. Final Addition: Using a standard adder (like a Ripple Carry Adder) to sum the final two rows into the result.

For this project, we chose the Dadda multiplier because it is a pure combinational circuit, making it an ideal candidate to demonstrate the ATPG algorithm D without the complexity of sequential scan chains.

2 Project Workflow

The workflow adopted for this project is as follows:

1. **RTL Design Simulation:** Verifying functional correctness using Icarus Verilog and GTKWave.
2. **Logic Synthesis:** Converting the behavioral RTL into a gate-level netlist using Yosys.
3. **ATPG:** Generating test patterns and detecting faults using Atalanta.

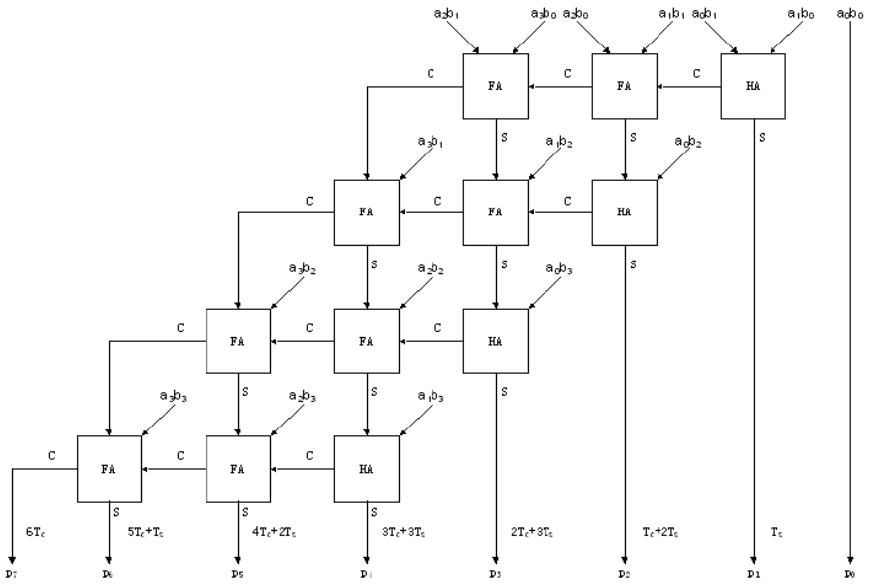

Figure 2: 4x4 Array Multiplier Circuit

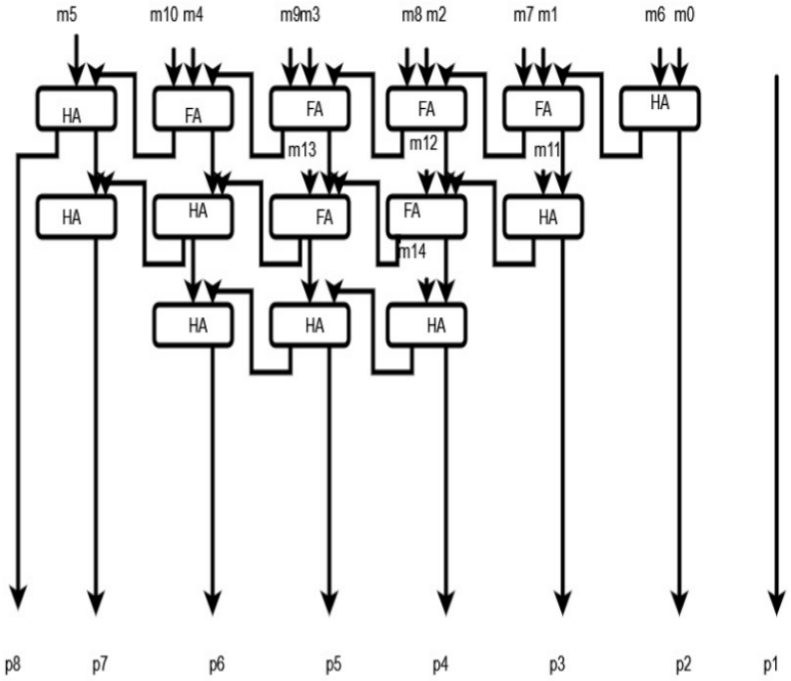

Figure 1: 4x4 Dadda Multiplier Circuit

3 RTL Design for 4-bit Dadda Multiplier

```
module dadda4(
    input [3:0] A,
    input [3:0] B,
    output [7:0] y
    );

    wire pp00, pp01, pp02, pp03;
    wire pp10, pp11, pp12, pp13;
    wire pp20, pp21, pp22, pp23;
    wire pp30, pp31, pp32, pp33;

    assign pp00 = A[0] & B[0]; assign pp01 = A[1] & B[0]; assign pp02 = A[2] & B[0];
    assign pp10 = A[0] & B[1]; assign pp11 = A[1] & B[1]; assign pp12 = A[2] & B[1];
    assign pp20 = A[0] & B[2]; assign pp21 = A[1] & B[2]; assign pp22 = A[2] & B[2];
    assign pp30 = A[0] & B[3]; assign pp31 = A[1] & B[3]; assign pp32 = A[2] & B[3];

    // Stage 1 (Reduction)
    wire s1_3, c1_4;
    // HA at Column 3
    assign s1_3 = pp30 ^ pp21;
    assign c1_4 = pp30 & pp21;

    // Stage 2 (Reduction)
    wire s2_2, c2_3;
    wire s2_3, c2_4;
    wire s2_4, c2_5;

    // FA at Col 2
    assign s2_2 = pp20 ^ pp11 ^ pp02;
    assign c2_3 = (pp20 & pp11) | (pp20 & pp02) | (pp11 & pp02);

    // FA at Col 3
    assign s2_3 = s1_3 ^ pp12 ^ pp03;
    assign c2_4 = (s1_3 & pp12) | (s1_3 & pp03) | (pp12 & pp03);

    // FA at Col 4
    assign s2_4 = pp31 ^ pp22 ^ pp13;
    assign c2_5 = (pp31 & pp22) | (pp31 & pp13) | (pp22 & pp13);

    // Stage 3 (Reduction)
    wire s3_4, c3_5;
    wire s3_5, c3_6;

    // FA at Col 4
    assign s3_4 = s2_4 ^ c1_4 ^ c2_4;
```

```
    assign c3_5 = (s2_4 & c1_4) | (s2_4 & c2_4) | (c1_4 & c2_4);

    // FA at Col 5
    assign s3_5 = pp32 ^ pp23 ^ c2_5;
    assign c3_6 = (pp32 & pp23) | (pp32 & c2_5) | (pp23 & c2_5);

    // Final Ripple Carry Adder
    wire cf2, cf3, cf4, cf5, cf6;

    assign y[0] = pp00;
    assign y[1] = pp10 ^ pp01;
    assign cf2  = pp10 & pp01;

    assign y[2] = s2_2 ^ cf2;
    assign cf3  = s2_2 & cf2;

    assign y[3] = s2_3 ^ c2_3 ^ cf3;
    assign cf4 = (s2_3 & c2_3) | (s2_3 & cf3) | (c2_3 & cf3);

    assign y[4] = s3_4 ^ cf4;
    assign cf5  = s3_4 & cf4;

    assign y[5] = s3_5 ^ c3_5 ^ cf5;
    assign cf6  = (s3_5 & c3_5) | (s3_5 & cf5) | (c3_5 & cf5);

    // y6 (FA) - Fixed Logic
    assign y[6] = pp33 ^ c3_6 ^ cf6;

    assign y[7] = (pp33 & c3_6) | (pp33 & cf6) | (c3_6 & cf6);

endmodule
```

4 Simulation: Icarus Verilog & GTKWave

4.1 Tool Usage

```
iverilog -o dadda_sim dadda4.v dadda4_tb.v

vvp dadda_sim

gtkwave dump.vcd
```

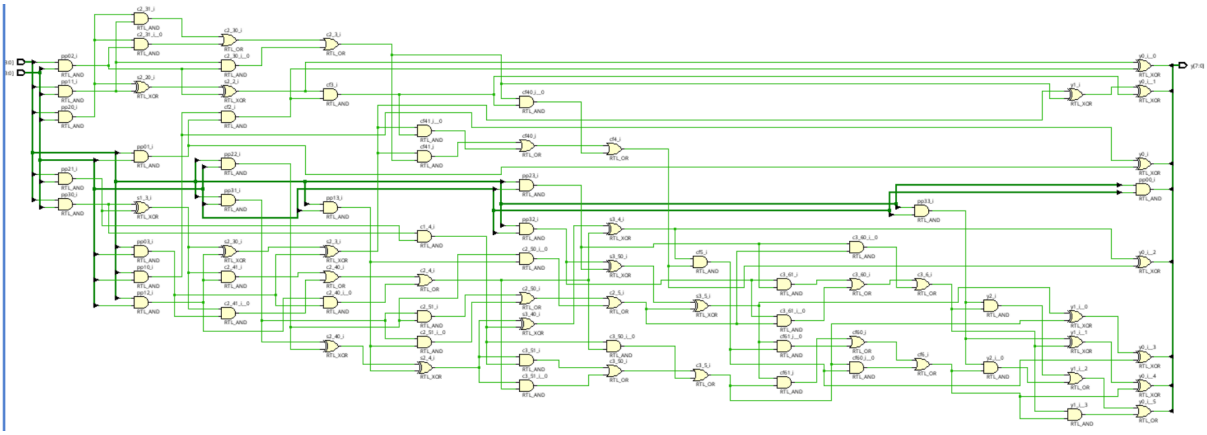

Figure 3: The above synthesized design was generated on Vivado 2023.2

Table 1: Synthesized Schematic Summary

Metric	Count	Single Stuck At Faults	Fault Collapse
Primary Input	8	$8 \times 1 = 8$	tofill
Primary Output	8	$8 \times 1 = 8$	tofill
Primitive AND Gate	44	$44 \times 6 = 264$	tofill
Primitive OR Gate	16	$16 \times 6 = 96$	tofill
Primitive NOT Gate	0	0	tofill
Primitive XOR Gate	20	$20 \times 12 \times 2 = 480$	tofill
Fan Out Stems	40	$40 \times 3 = 120$	tofill

Total number of single stuck-at faults $= 8 + 8 + 264 + 96 + 0 + 480 + 120 = 956$

4.2 Testbench

```
‘timescale 1ns / 1ps

module tb_dadda4;

    reg [3:0] A;
    reg [3:0] B;


    wire [7:0] y;


    dadda4 uut (
        .A(A),
        .B(B),
        .y(y)
    );


    initial begin
        $dumpfile("dadda4_wave.vcd");
        $dumpvars(0, tb_dadda4);
    end

    initial begin

$monitor("Time: %0t ns | A = %d | B = %d | y (Result) = %d", $time, A, B, y);


        A = 4’b0000; B = 4’b0000;


        #10 A = 4’d2; B = 4’d3;


        #10 A = 4’d7; B = 4’d5;


        #10 A = 4’d15; B = 4’d15;


        #10 A = 4’d9; B = 4’d4;


        #10 A = 4’d0; B = 4’d10;
```

```
        #10 A = 4'd11; B = 4'd6;


        #10 A = 4'd8; B = 4'd8;


        #10 A = 4'd12; B = 4'd3;


        #10 A = 4'd14; B = 4'd2;

        #10;
        $finish;
    end

endmodule
```

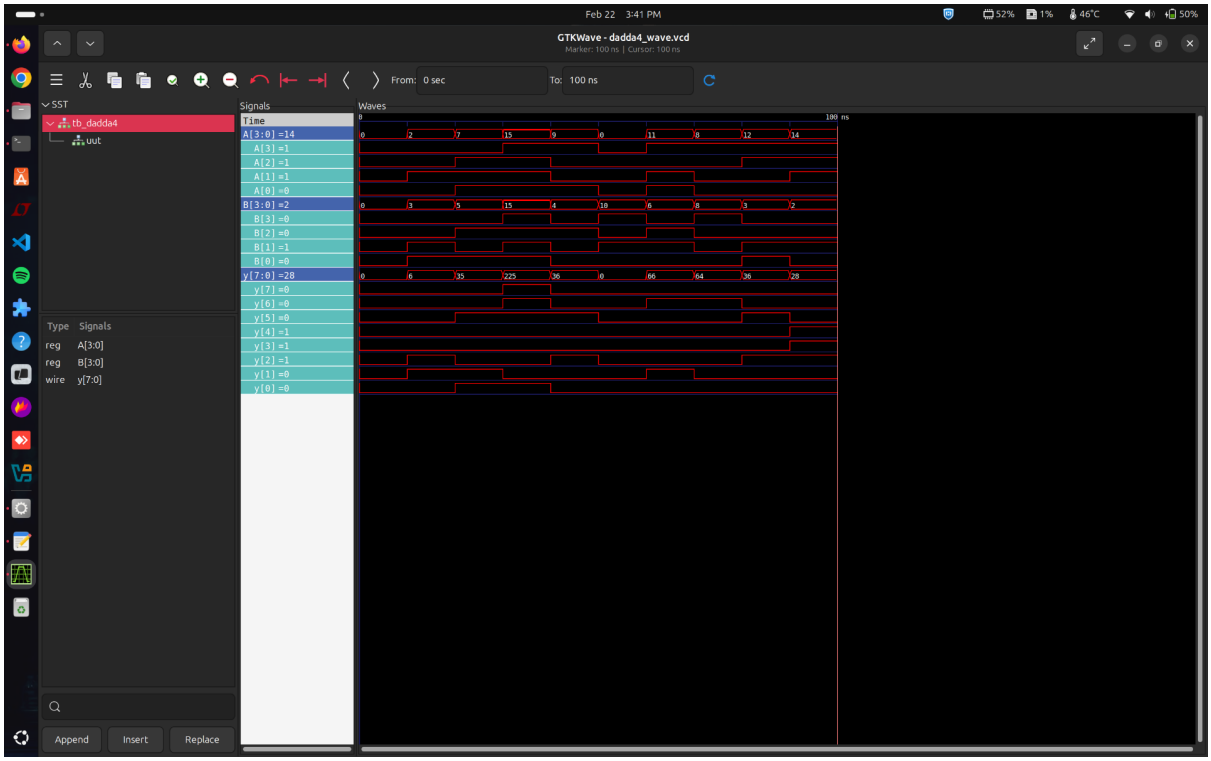

Figure 4: Simulation of 4 bit Dadda Multiplier

```
amrut@Maverick:~/Atalanta$ cd ..
amrut@Maverick:~$ vim four_bit_dadda_multiplier.v
amrut@Maverick:~$ vim four_bit_dadda_multiplier_tb.v
amrut@Maverick:~$ iverilog four_bit_dadda_multiplier.v four_bit_dadda_multiplier_tb.v -o sim
amrut@Maverick:~$ vvp sim
VCD info: dumpfile dadda4_wave.vcd opened for output.
Time: 0 ns | A =  0 | B =  0 | y (Result) =   0
Time: 10000 ns | A =  2 | B =  3 | y (Result) =   6
Time: 20000 ns | A =  7 | B =  5 | y (Result) =  35
Time: 30000 ns | A = 15 | B = 15 | y (Result) = 225
Time: 40000 ns | A =  9 | B =  4 | y (Result) =  36
Time: 50000 ns | A =  0 | B = 10 | y (Result) =   0
Time: 60000 ns | A = 11 | B =  6 | y (Result) =  66
Time: 70000 ns | A =  8 | B =  8 | y (Result) =  64
Time: 80000 ns | A = 12 | B =  3 | y (Result) =  36
Time: 90000 ns | A = 14 | B =  2 | y (Result) =  28
four_bit_dadda_multiplier_tb.v:62: $finish called at 100000 (1ps)
amrut@Maverick:~$ ls
```

Figure 5: TestBench Results

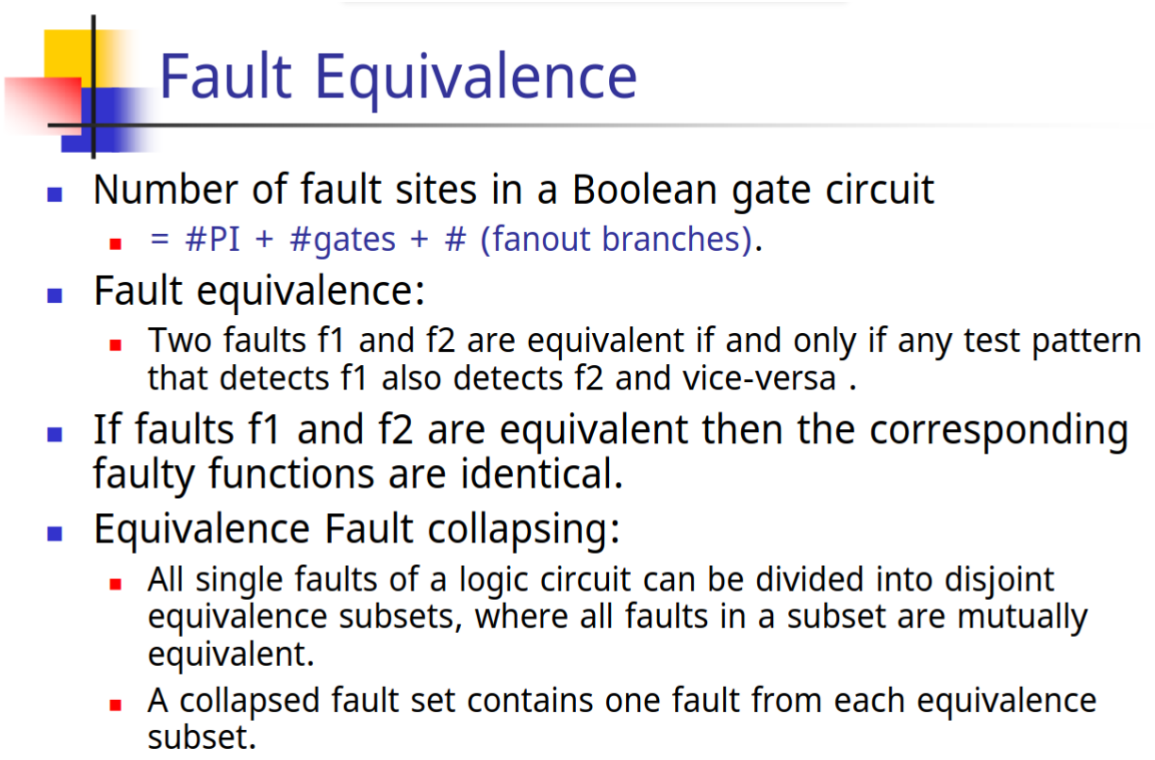

Figure 6: Fault Equivalence

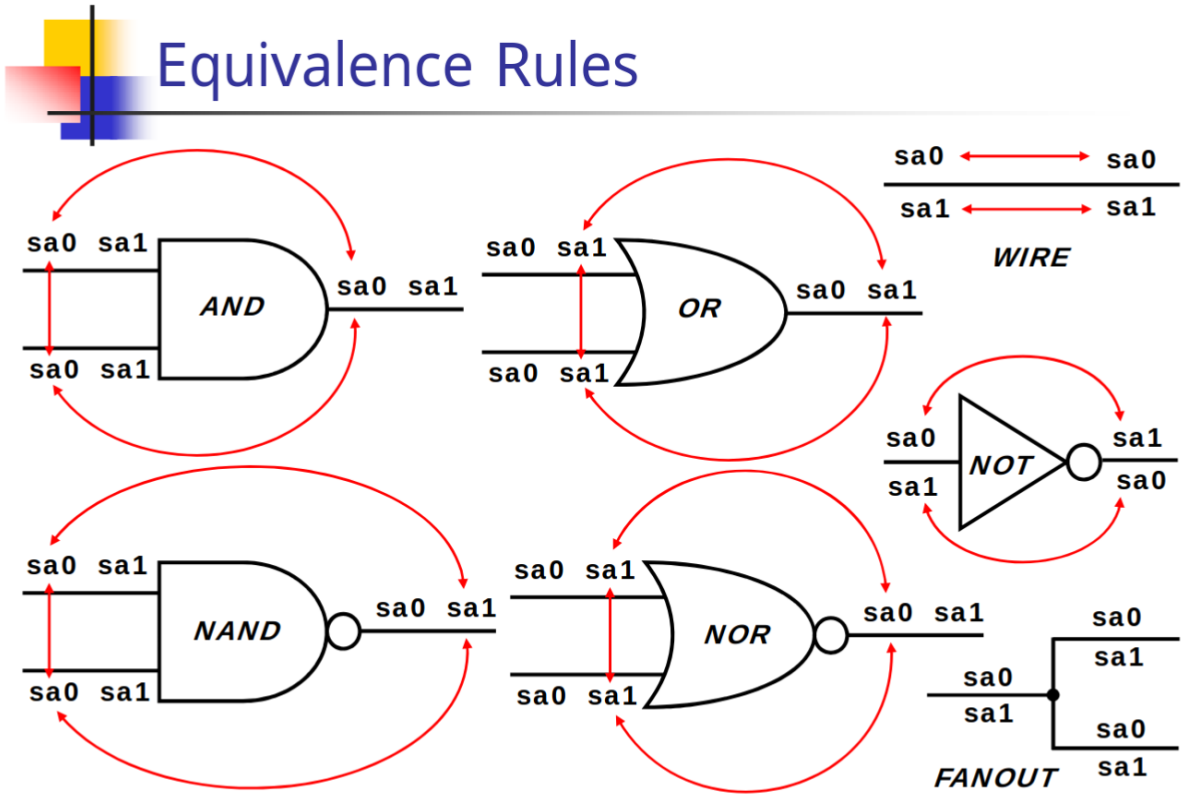

Figure 7: Fault Equivalence for Gates

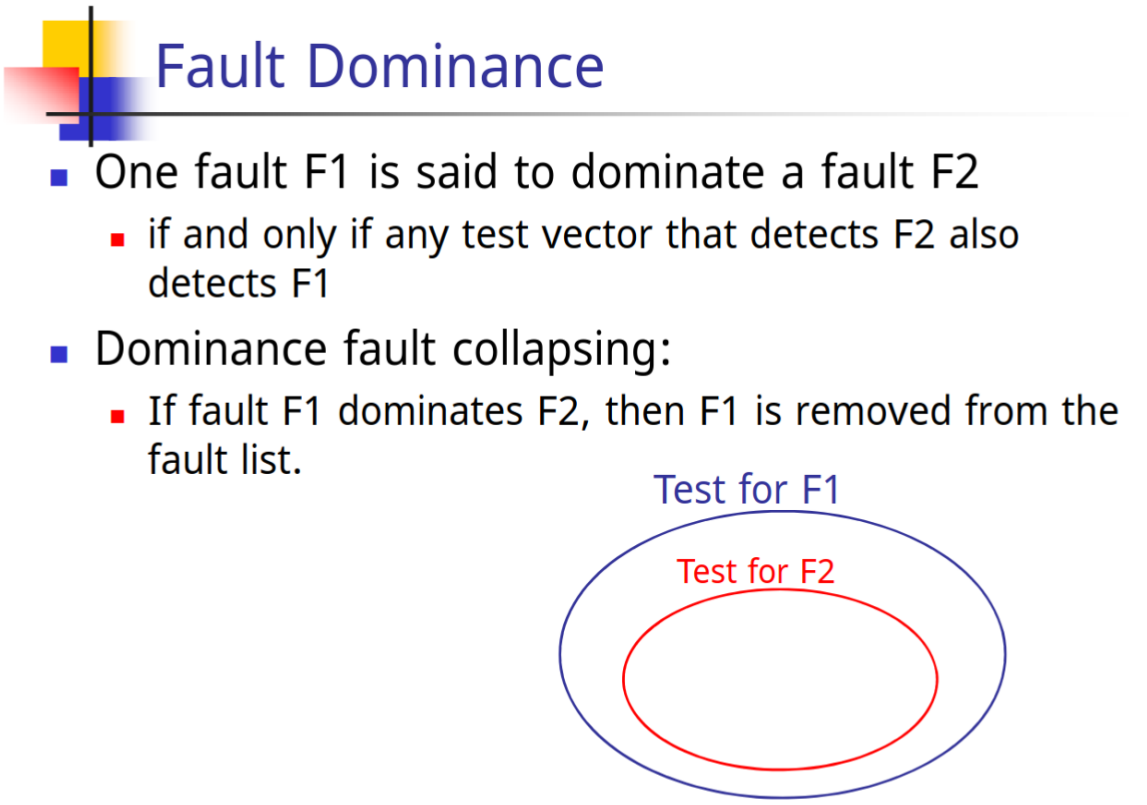

Figure 8: Fault Dominance

An n-input elementary gate has n+1 non-dominating faults in its minimal length dominance-reduced fault list.

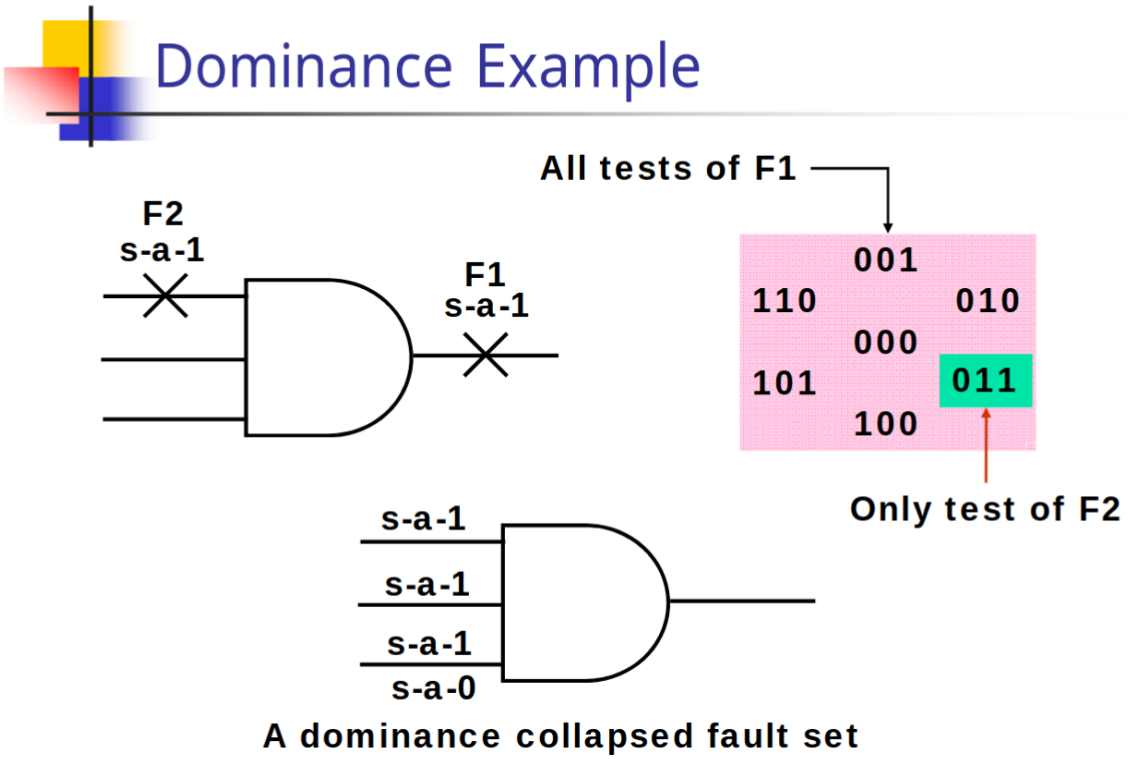

Figure 9: Fault Dominance example

5 Yosys : Synthesis

5.1 Synth Script

```
read_verilog dadda4bit.v
hierarchy -check -top dadda4
proc  ---(1)
opt  ---(2)
techmap  ---(3)
synth -top dadda4 ---(3)
write_verilog dadda4synth.v
write_blif dadd4blifed.blif
```

What's a blif file?

A BLIF (Berkeley Logic Interchange Format) file is a simple, standard file format used for exchanging sequential logic designs between different electronic design automation (EDA) programs. In the context of Yosys synthesis, the BLIF file serves as an output format for the synthesized netlist of a digital circuit, which can then be used by other downstream tools.

Also, performed using a different synth script:

```
read_verilog dadda4bit.v
hierarchy -check -top dadda4
read_verilog -lib cmos_cells.v
synth
dfflibmap -liberty cmos_cells.lib
clear
abc -liberty cmos_cells.lib
opt_clean
write_verilog dadda4synthesized.v
```

5.2 Netlist Obtained

```
(* top =  1  *)
(* src = "dadda4bit.v:1.1-76.10" *)
module dadda4(A, B, y);
  wire _000_;
  wire _001_;
  wire _002_;
  wire _003_;
  wire _004_;
  wire _005_;
  wire _006_;
  wire _007_;
  wire _008_;
  wire _009_;
  wire _010_;
  wire _011_;
  wire _012_;
  wire _013_;
  wire _014_;
  wire _015_;
  wire _016_;
  wire _017_;
  wire _018_;
  wire _019_;
  wire _020_;
  wire _021_;
  wire _022_;
  wire _023_;
  wire _024_;
  wire _025_;
  wire _026_;
  wire _027_;
  wire _028_;
  wire _029_;
  wire _030_;
  wire _031_;
  wire _032_;
  wire _033_;
  wire _034_;
  wire _035_;
  wire _036_;
  wire _037_;
  wire _038_;
  wire _039_;
  wire _040_;
  wire _041_;
  wire _042_;
```

```
wire _043_;
wire _044_;
wire _045_;
wire _046_;
wire _047_;
wire _048_;
wire _049_;
wire _050_;
wire _051_;
wire _052_;
wire _053_;
wire _054_;
wire _055_;
wire _056_;
wire _057_;
wire _058_;
wire _059_;
wire _060_;
wire _061_;
wire _062_;
wire _063_;
wire _064_;
wire _065_;
wire _066_;
wire _067_;
wire _068_;
wire _069_;
wire _070_;
wire _071_;
(* src = "dadda4bit.v:2.17-2.18" *)
input [3:0] A;
wire [3:0] A;
(* src = "dadda4bit.v:3.17-3.18" *)
input [3:0] B;
wire [3:0] B;
(* src = "dadda4bit.v:7.10-7.14" *)
wire pp00;
(* src = "dadda4bit.v:4.18-4.19" *)
output [7:0] y;
wire [7:0] y;
assign pp00 = B[0] & A[0];
assign _000_ = ~(B[1] & A[0]);
assign _001_ = ~(A[1] & B[0]);
assign y[1] = _001_ ^ _000_;
assign _002_ = ~(B[2] & A[0]);
assign _003_ = B[1] & A[1];
assign _004_ = _003_ ^ _002_;
assign _005_ = ~(A[2] & B[0]);
```

```
assign _006_ = ~(_005_ ^ _004_);
assign _007_ = _001_ | _000_;
assign y[2] = _007_ ^ _006_;
assign _008_ = ~(B[3] & A[0]);
assign _009_ = B[2] & A[1];
assign _010_ = _009_ ^ _008_;
assign _011_ = B[1] & A[2];
assign _012_ = _011_ ^ _010_;
assign _013_ = ~(A[3] & B[0]);
assign _014_ = _013_ ^ _012_;
assign _015_ = _003_ & ~(_002_);
assign _016_ = _005_ | _002_;
assign _017_ = _016_ & ~(_015_);
assign _018_ = _003_ & ~(_005_);
assign _019_ = _018_ | ~(_017_);
assign _020_ = ~(_019_ ^ _014_);
assign _021_ = _007_ | _006_;
assign y[3] = _021_ ^ _020_;
assign _022_ = ~(B[3] & A[1]);
assign _023_ = B[2] & A[2];
assign _024_ = _023_ ^ _022_;
assign _025_ = ~(B[1] & A[3]);
assign _026_ = _025_ ^ _024_;
assign _027_ = _009_ & ~(_008_);
assign _028_ = _027_ ^ _026_;
assign _029_ = _011_ & ~(_010_);
assign _030_ = _013_ | _010_;
assign _031_ = _030_ & ~(_029_);
assign _032_ = _011_ & ~(_013_);
assign _033_ = _031_ & ~(_032_);
assign _034_ = _033_ ^ _028_;
assign _035_ = ~(_019_ & _014_);
assign _036_ = _014_ & ~(_021_);
assign _037_ = _035_ & ~(_036_);
assign _038_ = _019_ & ~(_021_);
assign _039_ = _037_ & ~(_038_);
assign y[4] = _039_ ^ _034_;
assign _040_ = B[3] & A[2];
assign _041_ = B[2] & A[3];
assign _042_ = ~(_041_ ^ _040_);
assign _043_ = _023_ & ~(_022_);
assign _044_ = _025_ | _022_;
assign _045_ = _044_ & ~(_043_);
assign _046_ = _023_ & ~(_025_);
assign _047_ = _045_ & ~(_046_);
assign _048_ = _047_ ^ _042_;
assign _049_ = ~(_027_ & _026_);
assign _050_ = _026_ & ~(_033_);
```

```
  assign _051_ = _049_ & ~(_050_);
  assign _052_ = _027_ & ~(_033_);
  assign _053_ = _052_ | ~(_051_);
  assign _054_ = ~(_053_ ^ _048_);
  assign _055_ = _039_ | _034_;
  assign y[5] = _055_ ^ _054_;
  assign _056_ = B[3] & A[3];
  assign _057_ = _041_ & _040_;
  assign _058_ = _040_ & ~(_047_);
  assign _059_ = ~(_058_ | _057_);
  assign _060_ = _041_ & ~(_047_);
  assign _061_ = _059_ & ~(_060_);
  assign _062_ = _061_ ^ _056_;
  assign _063_ = ~(_053_ & _048_);
  assign _064_ = _048_ & ~(_055_);
  assign _065_ = _063_ & ~(_064_);
  assign _066_ = _053_ & ~(_055_);
  assign _067_ = _065_ & ~(_066_);
  assign y[6] = _067_ ^ _062_;
  assign _068_ = _056_ & ~(_061_);
  assign _069_ = _056_ & ~(_067_);
  assign _070_ = _069_ | _068_;
  assign _071_ = ~(_067_ | _061_);
  assign y[7] = _071_ | _070_;
  assign y[0] = pp00;
endmodule
```

6 ATPG: Atalanta

We used Atalanta to run the multiplier circuit bench file.

```
#
INPUT(A0)
INPUT(A1)
INPUT(A2)
INPUT(A3)
INPUT(B0)
INPUT(B1)
INPUT(B2)
INPUT(B3)

OUTPUT(y0)
OUTPUT(y1)
OUTPUT(y2)
OUTPUT(y3)
OUTPUT(y4)
OUTPUT(y5)
OUTPUT(y6)
OUTPUT(y7)

#
# Partial products
#
pp00 = AND(A0, B0)
pp01 = AND(A1, B0)
pp02 = AND(A2, B0)
pp03 = AND(A3, B0)

pp10 = AND(A0, B1)
pp11 = AND(A1, B1)
pp12 = AND(A2, B1)
pp13 = AND(A3, B1)

pp20 = AND(A0, B2)
pp21 = AND(A1, B2)
pp22 = AND(A2, B2)
pp23 = AND(A3, B2)

pp30 = AND(A0, B3)
pp31 = AND(A1, B3)
pp32 = AND(A2, B3)
pp33 = AND(A3, B3)

#
# Stage 1 { Column 3 (Half Adder)
#
```

```
s1_3 = XOR(pp30, pp21)
c1_4 = AND(pp30, pp21)

#
# Stage 2 { Column 2 (Full Adder)
#
xor_c2_1 = XOR(pp20, pp11)
s2_2     = XOR(xor_c2_1, pp02)

and_c2_1 = AND(pp20, pp11)
and_c2_2 = AND(pp20, pp02)
and_c2_3 = AND(pp11, pp02)
or_c2_1  = OR(and_c2_1, and_c2_2)
c2_3     = OR(or_c2_1, and_c2_3)

#
# Stage 2 { Column 3 (Full Adder)
#
xor_c3_1 = XOR(s1_3, pp12)
s2_3     = XOR(xor_c3_1, pp03)

and_c3_1 = AND(s1_3, pp12)
and_c3_2 = AND(s1_3, pp03)
and_c3_3 = AND(pp12, pp03)
or_c3_1  = OR(and_c3_1, and_c3_2)
c2_4     = OR(or_c3_1, and_c3_3)

#
# Stage 2 { Column 4 (Full Adder)
#
xor_c4_1 = XOR(pp31, pp22)
s2_4     = XOR(xor_c4_1, pp13)

and_c4_1 = AND(pp31, pp22)
and_c4_2 = AND(pp31, pp13)
and_c4_3 = AND(pp22, pp13)
or_c4_1  = OR(and_c4_1, and_c4_2)
c2_5     = OR(or_c4_1, and_c4_3)

#
# Stage 3 { Column 4 (Full Adder)
#
xor_s3c4_1 = XOR(s2_4, c1_4)
s3_4       = XOR(xor_s3c4_1, c2_4)

and_s3c4_1 = AND(s2_4, c1_4)
and_s3c4_2 = AND(s2_4, c2_4)
and_s3c4_3 = AND(c1_4, c2_4) ---------------------------------(3)
```

```
or_s3c4_1  = OR(and_s3c4_1, and_s3c4_2)
c3_5       = OR(or_s3c4_1, and_s3c4_3)

#
# Stage 3 { Column 5 (Full Adder)
#
xor_s3c5_1 = XOR(pp32, pp23)
s3_5       = XOR(xor_s3c5_1, c2_5)

and_s3c5_1 = AND(pp32, pp23)
and_s3c5_2 = AND(pp32, c2_5)
and_s3c5_3 = AND(pp23, c2_5)
or_s3c5_1  = OR(and_s3c5_1, and_s3c5_2)
c3_6       = OR(or_s3c5_1, and_s3c5_3)

#
# Final Ripple-Carry Addition (Columns 0 to 7)
#

# Column 0
y0 = BUF(pp00)

# Column 1
y1  = XOR(pp10, pp01)
cf2 = AND(pp10, pp01)

# Column 2
y2  = XOR(s2_2, cf2)
cf3 = AND(s2_2, cf2)

# Column 3 (Full Adder logic)
y3_tmp1  = XOR(s2_3, c2_3)
y3       = XOR(y3_tmp1, cf3)

and_f3_1 = AND(s2_3, c2_3)
and_f3_2 = AND(s2_3, cf3)
and_f3_3 = AND(c2_3, cf3)
or_f3_1  = OR(and_f3_1, and_f3_2)
cf4      = OR(or_f3_1, and_f3_3)

# Column 4 (Half Adder: s3_4 + cf4)
# Note: In standard Dadda 4x4, this stage reduces to 2 bits.
y4  = XOR(s3_4, cf4)
cf5 = AND(s3_4, cf4)

# Column 5 (Full Adder: s3_5 + c3_5 + cf5)
y5_tmp1  = XOR(s3_5, c3_5)
y5       = XOR(y5_tmp1, cf5)
```

```
and_f5_1 = AND(s3_5, c3_5)
and_f5_2 = AND(s3_5, cf5)
and_f5_3 = AND(c3_5, cf5) ---------------------------------(2)
or_f5_1  = OR(and_f5_1, and_f5_2)
cf6      = OR(or_f5_1, and_f5_3)

# Column 6 (Full Adder: pp33 + c3_6 + cf6)
y6_tmp1 = XOR(pp33, c3_6)
y6      = XOR(y6_tmp1, cf6)

# Carry to Column 7 (Full Adder Carry Logic)
tmp_and1_f6 = AND(pp33, c3_6)
tmp_and2_f6 = AND(pp33, cf6)
tmp_or1_f6  = OR(tmp_and1_f6, tmp_and2_f6)
tmp_and3_f6 = AND(c3_6, cf6) ---------------------------------(1)
y7          = OR(tmp_or1_f6, tmp_and3_f6)
```

Atalanta File Directory:

```
~/Atalanta/
|
|_____dadda_4bit.v
|
|_____dadda_4bit_tb.v
|
|_____dadda.vcd
|
|_____dadda_4bit_synthesized.v
|
|_____dadda_4bit.bench
|
|_____dadda_4bit.test
|
|_____dadda_4bit.ufaults
|
|_____dadda_4bit.flt
```

```
amrut@Maverick:~/Atalanta$ cat dadda-multiplier-fourbit.test
* Name of circuit:  dadda4bit.bench
* Primary inputs :
  A0 A1 A2 A3 B0 B1 B2 B3

* Primary outputs:
  y0 y1 y2 y3 y4 y5 y6 y7


* Test patterns and fault free responses:

                    1: 11011100 10000100
                    2: 10101111 11010010
                    3: 11101100 10101000
                    4: 00110011 00001001
                    5: 00101000 00100000
                    6: 10011111 11100001
                    7: 11010111 01011001
                    8: 01101100 01001000
                    9: 01100011 00010010
                   10: 11000011 00100100
                   11: 11111111 10000111
                   12: 11001100 10010000
                   13: 01010101 00100110
                   14: 10111111 11000011
amrut@Maverick:~/Atalanta$ cat dadda4bit.vec
                 11011100
                 10101111
                 11101100
                 00110011
                 00101000
                 10011111
                 11010111
                 01101100
                 01100011
                 11000011
                 11111111
                 11001100
                 01010101
                 10111111
END
amrut@Maverick:~/Atalanta$ cat dadda4bit.ufaults
                 tmp_and3_f6 /0
                 and_f5_3 /0
                 and_s3c4_3 /0
```

6.1 Test Report Analysis

```
1. Circuit structure
   Name of the circuit                        : 4-bit_Dadda_multiplier
   Number of primary inputs                   : 8
   Number of primary outputs                  : 8
   Number of gates                            : 81
   Level of the circuit                       : 14

2. ATPG parameters
   Test pattern generation Mode               : RPT + DTPG + TC -------------(1)
   Limit of random patterns (packets)         : 16
   Backtrack limit                            : 10
   Initial random number generator seed       : 1771760837
   Test pattern compaction Mode               : REVERSE + SHUFFLE
   Limit of shuffling compaction              : 2
   Number of shuffles                         : 8

3. Test pattern generation results
   Number of test patterns before compaction : 31 -------------(2)
   Number of test patterns after compaction  : 15 -------------(3)
   Fault coverage                             : 98.986 % -------------(4)
   Number of collapsed faults                 : 296 -------------(5)
   Number of identified redundant faults      : 3 -------------(6)
   Number of aborted faults                   : 0
   Total number of backtrackings              : 11

4. Memory used                                : 0.000 MB

5. CPU time
   Initialization                             : 0.000 Secs
   Fault simulation                           : 0.000 Secs
   FAN                                        : 0.000 Secs
   Total                                      : 0.000 Secs
```

These are the untestable faults

```
amrut@Maverick:~/Atalanta$ cat daddamulfault.flt
        and_f5_3 /0
        and_s3c4_3 /0
        tmp_and3_f6 /0


******   SUMMARY OF TEST PATTERN GENERATION RESULTS   ******
1. Circuit structure
   Name of the circuit                       :
   Number of primary inputs                  : 8
   Number of primary outputs                 : 8
   Number of gates                           : 81
   Level of the circuit                      : 14

2. ATPG parameters
   Test pattern generation Mode              : RPT + DTPG + TC
   Limit of random patterns (packets)        : 16
   Backtrack limit                           : 10
   Initial random number generator seed      : 1771768406
   Test pattern compaction Mode              : REVERSE + SHUFFLE
   Limit of shuffling compaction             : 2
   Number of shuffles                        : 16777216

3. Test pattern generation results
   Number of test patterns before compaction : 0
   Number of test patterns after compaction  : 0
   Fault coverage                            : 0.000 %
   Number of collapsed faults                : 3
   Number of identified redundant faults     : 3
   Number of aborted faults                  : 0
   Total number of backtrackings             : 11

4. Memory used                               : 0.000 MB

5. CPU time
   Initialization                            : 0.000 Secs
   Fault simulation                          : 0.000 Secs
   FAN                                       : 0.000 Secs
   Total                                     : 0.000 Secs
```

For stuck at 0 fault at pp01, we are hand tracing using D algorithm

```
amrut@Maverick:~/Atalanta$ cat daddamulfault.flt
        pp01 /0


amrut@Maverick:~/Atalanta$ cat dadda4bit.vec
        01001000
END
```

D-algorithm :

1. Activate the fault
2. Propagate the fault
3. Justify the fault

Table 2: D algorithm applied on chosen fault

Rule	**A0**	**A1**	**A2**	**A3**	**B0**	**B1**	**B2**	**B3**	**PP01 (SA0)**	**PP10**	**Y1**
Activation	–	1	–	–	1	–	–	–	D	–	–
Propagation	0/1	1	–	–	1	0/1	–	–	D	0/1	–
Detection	0/1	1	–	–	1	0/1	–	–	D	0/1	D/$\tilde{\text{D}}$

A0A1A2A3 B0B1B2B3 is {0, 1, x, x, 1, 0, x, x} or {1, 1, x, x, 1, 1, x, x}
The tool obtained : 01001000
Which matches with our above result

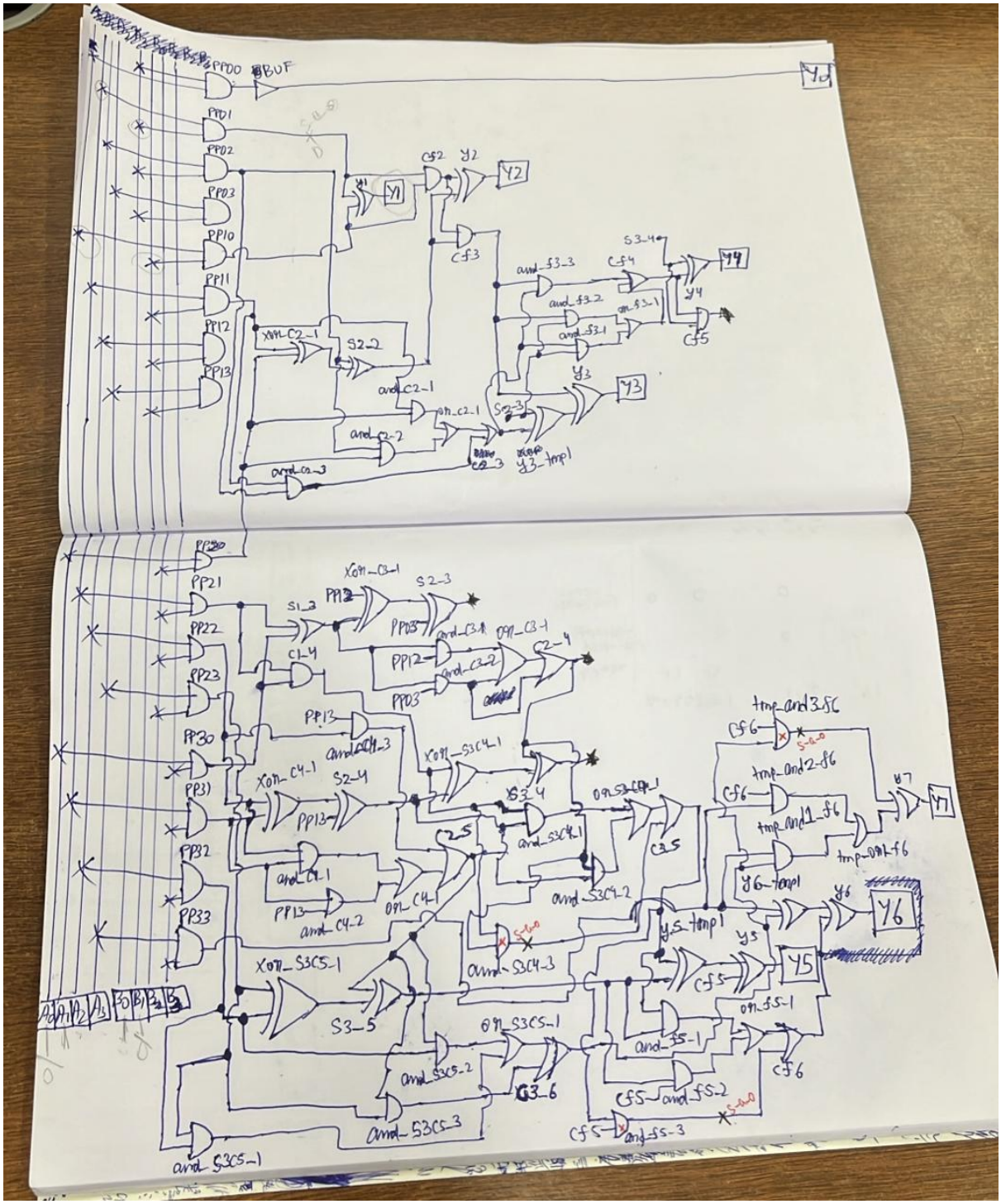

Figure 10: Hand Tracing using D algorithm

7 ATPG: Atalanta

Utilized Atalanta, a tool for ATPG for combinational circuits using the FAN algorithm.

7.1 Tool Commands

```
# Run Atalanta with Fault list (-f), Test output (-t), and Benchmark netlist (-v)
atalanta -f daddafault.flt -t daddatest.test -v daddabench.bench
```

- **.bench**: The gate-level netlist converted to bench format.
- **.test**: The output file that contains the test vectors generated.
- **.flt**: The output report listing detected stuck-at faults.